UNIVERSITÉ CÔTE D'AZUR

Inria

Étude Comparative des Systèmes de Fichiers pour SSD

Analyse architecturale, évaluation des performances et intégration de VeraCrypt

Auteur : BENSALLAH Younes

Encadrant : TOUATI Sid

Formation : Master 1 Informatique

Année : 2025–2026

Systèmes étudiés : JFFS • JFFS2 • UBIFS • F2FS • YAFFS2 • LogFS • VeraCrypt

12 Mai 2026

Résumé

Ce rapport présente une étude comparative des systèmes de fichiers conçus pour les supports de stockage flash (SSD), en étendant les travaux de Djerada Achour (janvier 2026) par l'intégration de VeraCrypt, un logiciel de chiffrement à la volée. Deux familles de systèmes sont examinées : les systèmes pour SSD avec FTL (F2FS) et les systèmes pour flash brute MTD (JFFS2, UBIFS). Les systèmes obsolètes JFFS, YAFFS2 et LogFS font l'objet d'une analyse théorique. Des benchmarks pratiques ont été conduits dans un environnement Kali Linux sous VirtualBox, en mesurant l'overhead disque, le nombre maximal de fichiers et répertoires, la taille maximale d'un fichier, et les performances lecture/écriture selon différentes tailles de blocs. Les résultats montrent qu'UBIFS surpasse JFFS2 de manière significative en écriture séquentielle, que VeraCrypt introduit un overhead de chiffrement modéré mais acceptable, et que F2FS présente un overhead de formatage plus élevé que les systèmes MTD.

Table des matières

1 Introduction **3**
- 1.1 Contexte . . . 3
- 1.2 Environnement matériel et virtuel . . . 3
- 1.3 Utilisation de l'intelligence artificielle . . . 4

2 État de l'Art, Architectures des Systèmes **4**
- 2.1 Rappel : mécanismes OS pertinents . . . 4
- 2.2 Systèmes pour flash brute MTD (actifs) . . . 4
  - 2.2.1 JFFS2 : Log-Structured pur . . . 4
  - 2.2.2 UBIFS : Log-Structured avec index persistant . . . 4
- 2.3 Systèmes pour SSD avec FTL (actifs) . . . 4
  - 2.3.1 F2FS : Flash-Friendly File System . . . 4
- 2.4 VeraCrypt : chiffrement à la volée . . . 5
- 2.5 Systèmes obsolètes - Analyse théorique . . . 5
  - 2.5.1 JFFS (1999) . . . 5
  - 2.5.2 YAFFS2 (2003) . . . 5
  - 2.5.3 LogFS (2009–2016) . . . 5

3 Méthodologie Expérimentale **6**
- 3.1 Environnement de test . . . 6
- 3.2 Mise en place des systèmes de fichiers . . . 6
  - 3.2.1 JFFS2 et UBIFS via block2mtd . . . 6
  - 3.2.2 F2FS et VeraCrypt . . . 6
- 3.3 Protocole de tests . . . 7
- 3.4 Limites méthodologiques . . . 7

4 Résultats **7**
- 4.1 Overhead disque à vide . . . 7
- 4.2 Nombre maximal de fichiers et répertoires . . . 8
- 4.3 Taille maximale d'un fichier . . . 9
- 4.4 Overhead VeraCrypt selon la taille de cluster . . . 9
- 4.5 Performances séquentielles selon la taille de blocs . . . 9

5 Discussion **11**
- 5.1 Analyse des performances d'écriture . . . 11
- 5.2 Analyse des performances de lecture . . . 11
- 5.3 Comparaison avec les résultats d'Achour . . . 12
- 5.4 Vue d'ensemble comparative . . . 13
- 5.5 Positionnement de VeraCrypt parmi les systèmes de fichiers . . . 13

6 Conclusion **13**
6.1 Synthèse des contributions 13
6.2 Recommandations 14
6.3 Perspectives 14

1 Introduction

1.1 Contexte

Le cours d'OS avancés [1] définit un système de fichiers comme *une implémentation par l'OS des opérations sur les fichiers (ouverture, copie, modification, écriture, accès, protection), une organisation des données sur le disque invisible à l'utilisateur, et une organisation logique visible à l'utilisateur.* Ce projet est une application directe de cette définition dans le contexte spécifique des mémoires flash NAND.

Les mémoires flash NAND constituent aujourd'hui le support dominant du stockage non-volatile, des cartes embarquées aux SSD NVMe haute performance. Leurs propriétés physiques (temps d'accès quasi-nul, asymétrie lecture/écriture, contrainte d'effacement par bloc et endurance limitée en cycles) imposent des adaptations logicielles profondes qui ont donné naissance à deux familles de systèmes de fichiers.

La première famille regroupe les systèmes conçus pour les SSD dotés d'un contrôleur FTL (*Flash Translation Layer*). Comme le souligne le cours [1], le contrôleur SSD utilise une technique de *wear levelling* pour équilibrer le nombre d'écritures par bloc, mécanisme *transparent à l'OS* : ext4, F2FS et XFS s'appuient sur cette transparence en déléguant entièrement la gestion de l'usure au contrôleur. La seconde famille, les systèmes pour flash brute MTD (*Memory Technology Device*) (JFFS2 et UBIFS), prend en charge directement ce mécanisme au niveau logiciel, sans couche d'abstraction matérielle.

Ce projet s'inscrit dans la continuité des travaux de Djerada Achour [3], qui a comparé ext4, F2FS, XFS, JFFS2 et UBIFS, en apportant trois contributions nouvelles : l'analyse architecturale et les benchmarks de **VeraCrypt**, l'étude de l'**overhead selon la taille des blocs** au formatage, et une méthodologie étendue mesurant le **nombre maximal de fichiers et répertoires**.

Ce projet couvre JFFS2, UBIFS et F2FS (testés pratiquement), VeraCrypt (testé pratiquement), et JFFS, YAFFS2, LogFS (analyse théorique uniquement, ces systèmes étant retirés ou absents du kernel mainline).

1.2 Environnement matériel et virtuel

Les expérimentations ont été conduites sur une machine hôte équipée d'un processeur **AMD Ryzen 7 3700X** (8 coeurs, 3,6 GHz) sous **Windows 11**, avec 16 Go de RAM DDR4. Le système utilisé est une machine virtuelle **Kali Linux 2021.4a** (noyau 6.1.0-kali7-amd64, architecture x86_64) exécutée sous VirtualBox, à laquelle 2 Go de RAM et 2 processeurs virtuels ont été alloués, sur un disque virtuel SATA de 80 Go (format VDI, mode normal). L'accélération matérielle KVM a été activée pour réduire l'overhead de virtualisation lors des opérations d'entrées/sorties.

Table 1 – Configuration matérielle et virtuelle

Composant	**Hôte (Windows 11)**	**VM (Kali Linux)**
Processeur	AMD Ryzen 7 3700X (3,6 GHz)	2 vCPU
RAM	16 Go DDR4	2 Go
Stockage	SSD (hôte)	VDI 80 Go (SATA)
OS	Windows 11	Kali Linux 2021
Noyau	/	kali7-amd64
Accélération	/	KVM, PAE/NX

1.3 Utilisation de l'intelligence artificielle

Dans le cadre de ce projet, l'assistant Claude Sonnet 4.6 a été utilisé pour la mise en forme du code de benchmark, la structuration du rapport en Latex, la correction linguistique ainsi que les scripts pour effectuer les benchmarks. L'ensemble de l'analyse technique, des benchmarks et des interprétations est le fruit d'un travail personnel.

2 État de l'Art, Architectures des Systèmes

2.1 Rappel : mécanismes OS pertinents

Trois mécanismes fondamentaux vus en cours d'OS [1] structurent l'analyse de ce rapport.

Les i-nodes sont des tables disque contenant les attributs d'un fichier et les adresses de ses blocs de données. Leur nombre est *fixé à la création de la partition*, constituant une limite absolue indépendante de l'espace disponible (voir Section 4.2).

Le cache disque Linux utilise toute la mémoire libre disponible pour la lecture, ce qui biaise les mesures en environnement virtualisé (voir Section 5.2).

La taille de bloc impacte directement les performances : trop petite elle multiplie les E/S, trop grande elle gaspille de l'espace [1].

2.2 Systèmes pour flash brute MTD (actifs)

2.2.1 JFFS2 : Log-Structured pur

JFFS2 (*Journalling Flash File System 2*), développé par Red Hat en 2001 sous licence GPL, adopte une architecture purement log-structured : toutes les écritures sont séquentielles, sans modification sur place. L'arborescence complète est reconstruite en RAM au montage, ce qui engendre un temps de montage en $O(n)$ et une consommation mémoire de 20–25% du volume. La taille maximale théorique d'un fichier est limitée à 4 Go (champ `dsize` sur 32 bits) et le volume recommandé ne dépasse pas 64 Mo, au-delà duquel les performances se dégradent sévèrement. JFFS2 gère directement le *wear leveling* et le *garbage collection* via un mécanisme de nœuds obsolètes. Le cours [1] note que JFFS2 présente de *mauvaises performances pour les grands SSD*, ce que nos benchmarks confirment quantitativement.

2.2.2 UBIFS : Log-Structured avec index persistant

UBIFS (*Unsorted Block Image File System*), introduit dans le kernel Linux 2.6.27 (2008) par Nokia et l'Université de Szeged, corrige les limitations de JFFS2. Il repose sur un index B+ tree persistant stocké sur le support, éliminant la reconstruction RAM au montage ($O(1)$) et réduisant la consommation mémoire à 1–2%. UBIFS s'appuie sur la couche UBI (*Unsorted Block Images*) pour le *wear leveling* explicite, ce que le cours [1] présente comme une couche d'abstraction entre le bloc logique et le bloc physique d'un SSD. La taille maximale théorique d'un fichier est 2^{63} octets.

2.3 Systèmes pour SSD avec FTL (actifs)

2.3.1 F2FS : Flash-Friendly File System

F2FS (*Flash-Friendly File System*), développé par Samsung et intégré dans Linux 3.8 (2013), adopte une architecture log-structured multi-tête avec 6 logs simultanés séparant les données par "température" (*Hot/Warm/Cold* × *Node/Data*). Cette organisation réduit l'amplification d'écriture en regroupant les données de durée de vie similaire. F2FS délègue le *wear leveling*

au contrôleur FTL du SSD, illustrant la transparence à l'OS décrite dans le cours [1]. La taille maximale d'un fichier est 3,94 To et celle du volume 16 To.

2.4 VeraCrypt : chiffrement à la volée

VeraCrypt, fork de TrueCrypt développé par IDRIX (France) et publié sous licence Apache 2.0, n'est pas un système de fichiers au sens défini dans le cours [1], mais une couche de chiffrement transparente qui s'intercale entre l'application et le système de fichiers hôte. Il opère en mode *On-The-Fly Encryption* (OTFE) : les données sont chiffrées/déchiffrées à la volée lors de chaque lecture/écriture, sans copie intermédiaire.

Architecture interne : VeraCrypt crée un volume conteneur dont les 512 premiers octets constituent un *header* chiffré contenant la clé maître, chiffré via PBKDF2 avec 327 661 à 655 331 itérations selon le hash choisi. Les données sont chiffrées en mode **XTS** (*XEX-based Tweaked CodeBook mode with ciphertext Stealing*), qui opère secteur par secteur (512 octets ou 4 096 octets), unité minimale correspondant au bloc disque défini dans le cours [1] (*minimal disk unit : disk block* $\geq$ *512B*). Les algorithmes supportés incluent AES-256, Serpent, Twofish, Camellia et Kuznyechik, ainsi que leurs cascades.

Impact sur les SSD : Le chiffrement XTS produit des données indistinguables du bruit aléatoire, neutralisant les algorithmes de compression des contrôleurs FTL. La commande TRIM ne peut pas traverser la couche de chiffrement sans exposer des métadonnées sur l'utilisation du volume.

Structure d'un volume VeraCrypt :

Zone	Contenu
Octets 0–511	Header chiffré (clé maître, algorithmes, taille)
Octets 512–65 535	Header de sauvegarde + données réservées
Octets 65 536 – fin	Données chiffrées (système de fichiers interne)

2.5 Systèmes obsolètes - Analyse théorique

2.5.1 JFFS (1999)

JFFS, premier système de fichiers Linux pour flash, adopte un log circulaire simple sans compression ni support NAND. Son *garbage collection* circulaire se dégrade sévèrement au-delà de quelques mégaoctets. Le cours [1] le mentionne comme utilisé par Red Hat pour les clés USB flash, usage désormais supplanté. Il a été remplacé par JFFS2 dès 2001.

2.5.2 YAFFS2 (2003)

YAFFS2 (*Yet Another Flash File System 2*), développé par Charles Manning, utilise la zone OOB de chaque page NAND pour stocker les métadonnées (Object ID, Chunk ID, Sequence Number, ECC). Le cours [1] le présente comme principalement destiné aux systèmes embarqués, non au Linux grand public. Il n'a jamais été intégré au kernel mainline en raison d'une double licence GPL/commerciale ambiguë.

2.5.3 LogFS (2009–2016)

LogFS, développé par Jörn Engel, visait à résoudre les limitations de JFFS2 pour les grands volumes (jusqu'à 16 To) via un *wandering tree*, l'objectif cité dans le cours [1] de *permettre de grandes SSD*. Il a été retiré du kernel en décembre 2016 faute de maintenance et d'adoption face à UBIFS.

Table 2 – Synthèse des limites théoriques de tous les systèmes

Système	Fichier max	Volume pratique	Montage	RAM	Statut
F2FS	3,94 To	16 To	$O(1)$	Faible	Actif
JFFS2	4 Go	64 Mo	$O(n)$	20–25%	Actif
UBIFS	8 Eo	Plusieurs To	$O(1)$	1–2%	Actif
VeraCrypt	1 Po	Volume hôte	$O(1)$	Faible	Actif
JFFS	4 Go	8 Mo	$O(n)$	Élevée	Obsolète
YAFFS2	512 Go	32 Go	$O(1)$	Moyenne	Non mainline
LogFS	8 Eo	16 To	$O(1)$	Faible	Abandonné

3 Méthodologie Expérimentale

3.1 Environnement de test

La configuration matérielle complète est détaillée en Section 1.2. Les systèmes de fichiers ont été montés sur des images fichiers créées dans le disque virtuel VDI de la VM Kali.

3.2 Mise en place des systèmes de fichiers

3.2.1 JFFS2 et UBIFS via block2mtd

Les systèmes JFFS2 et UBIFS nécessitent une interface MTD, absente sur un SSD standard. La solution adopté consiste à utiliser le module kernel `block2mtd` pour créer une interface MTD virtuelle sur un fichier image :

```
sudo modprobe jffs2 ubifs mtdblock block2mtd
dd if=/dev/zero of=~/resultats/mtd.img bs=1M count=256
sudo losetup /dev/loop0 ~/resultats/mtd.img
echo "/dev/loop0,131072" | sudo tee \
  /sys/module/block2mtd/parameters/block2mtd
# Montage JFFS2
sudo flash_erase /dev/mtd0 0 0
sudo mount -t jffs2 /dev/mtdblock0 /mnt/jffs2
# Montage UBIFS
sudo ubiformat /dev/mtd0 -y && sudo ubiattach -m 0 -d 0
sudo ubimkvol /dev/ubi0 -N vol -m
sudo mount -t ubifs ubi0:vol /mnt/ubifs
```

Cette approche vient mesurer les performances réelles sur le disque virtuel VDI.

3.2.2 F2FS et VeraCrypt

```
# F2FS
dd if=/dev/zero of=~/resultats/f2fs.img bs=1M count=512
sudo losetup /dev/loop1 ~/resultats/f2fs.img
sudo mkfs.f2fs /dev/loop1 && sudo mount -t f2fs /dev/loop1 /mnt/f2fs
# VeraCrypt
veracrypt --text --create ~/vc_test.vc --size=2G \
  --password=testpass123 --volume-type=normal \
  --encryption=AES --hash=SHA-512 --filesystem=ext4 \
  --non-interactive
sudo veracrypt --text --mount ~/vc_test.vc /mnt/vc \
  --password=testpass123 --non-interactive \
  --pim=0 --keyfiles="" --protect-hidden=no
```

3.3 Protocole de tests

Pour chaque système, cinq tests ont été conduits dans l'ordre suivant :

1. **Overhead disque à vide** : mesure de l'espace occupé immédiatement après le montage via `df -B1`.
2. **Nombre maximal de fichiers vides** : création itérative de fichiers (`touch`) jusqu'à erreur, avec mesure du débit.
3. **Nombre maximal de répertoires vides** : même protocole avec `mkdir`.
4. **Taille maximale d'un fichier** : écriture continue de zéros (`dd if=/dev/zero`) jusqu'à saturation du volume.
5. **Performances séquentielles selon la taille de blocs** : écriture et lecture avec `dd` pour les tailles 512B, 4K, 64K et 1M. L'option `conv=fdatasync` force la synchronisation physique pour éviter que le cache disque [1] ne fausse les mesures. Le cache de pages est invalidé entre écriture et lecture via la commande `echo 3 > /proc/sys/vm/drop_caches`.

Pour VeraCrypt, un test supplémentaire mesure l'overhead à vide pour trois tailles de cluster interne : 1 024, 2 048 et 4 096 octets.

3.4 Limites méthodologiques

Le disque virtuel VDI est un fichier sur le système hôte Windows, introduisant une couche d'abstraction supplémentaire. Comme le souligne le cours [2], la hiérarchie mémoire place le cache entre le processeur et la RAM : dans notre VM de 2 Go, le cache de pages Linux peut absorber l'intégralité des données écrites lors d'un benchmark de 256 Mo, biaisant les mesures de lecture à la hausse. Ces différences d'environnement expliquent les écarts observés entre nos résultats et ceux d'Achour.

4 Résultats

4.1 Overhead disque à vide

Table 3 – Overhead disque à vide après formatage

Système	Espace utilisé	Overhead	Explication
VeraCrypt (1K/2K/4K)	14 336 o	≈0,00%	Header chiffré non visible depuis l'intérieur
UBIFS	143 360 o	0,06%	Structures UBI + index B+ tree initial
JFFS2	5 926 912 o	2,21%	Superbloc + nœuds de journalisation
F2FS	119 545 856 o	22,35%	Sur-provisionnement (10%) + métadonnées

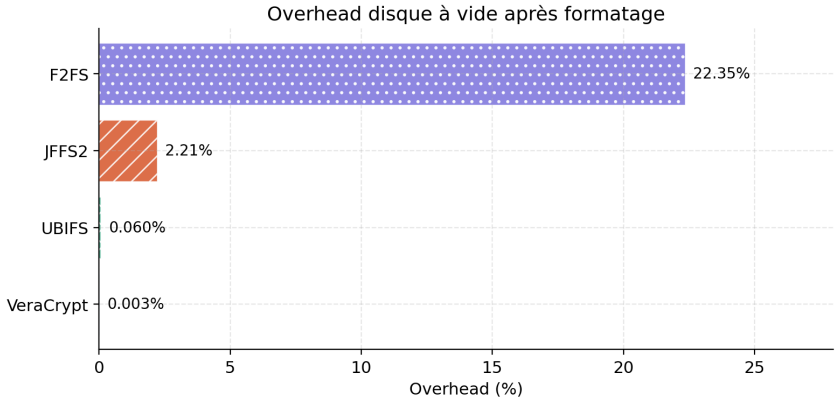


FIGURE 1 – Overhead disque à vide après formatage (%). Les écarts couvrent quatre ordres de grandeur entre VeraCrypt et F2FS.

L'overhead de F2FS (22,35%) est le plus élevé car le système réserve délibérément 10% du volume pour le sur-provisionnement du *garbage collection*, auquel s'ajoutent les structures de métadonnées (tables NAT, SIT, SSA et CP). L'overhead quasi-nul de VeraCrypt s'explique par l'invisibilité du header depuis l'intérieur du volume monté. L'overhead de JFFS2 (2,21%) correspond aux structures décrites dans le cours [1] : superbloc, bitmaps d'allocation et nœuds de journalisation.

4.2 Nombre maximal de fichiers et répertoires

TABLE 4 – Limites pratiques en nombre d'objets (volumes de test)

Système	**Fichiers max**	**Répertoires max**	**Facteur limitant**
VeraCrypt (2 Go)	131 060	131 060	Inodes ext4 alloués au formatage
JFFS2 (256 Mo)	$> 5\,000$	$> 5\,000$	Espace disque (test limité à 5 000)
UBIFS (256 Mo)	$> 5\,000$	$> 5\,000$	Espace disque (test limité à 5 000)
F2FS (512 Mo)	$> 5\,000$	$> 5\,000$	Espace disque (test limité à 5 000)

La limite de 131 060 fichiers/répertoires sur VeraCrypt est une conséquence directe du mécanisme d'i-node décrit dans le cours [1] : le nombre d'i-nodes est fixé à la création de la partition par `mkfs.ext4` (ratio par défaut : 1 inode / 16 Ko de volume). Pour un volume de 2 Go, cela donne exactement $2 \times 1024 \times 1024/16 = 131\,072$ inodes, dont 12 réservés pour les métadonnées système, soit 131 060 disponibles. Ce résultat illustre concrètement la limitation évoquée dans le cours : *un i-node est une table de taille maximale contenant un nombre limité d'adresses de blocs de données*. Pour JFFS2, UBIFS et F2FS, le test a été interrompu à 5 000 car JFFS2 présentait des temps de création prohibitifs (plus de 6 heures pour 500 000 fichiers).

4.3 Taille maximale d'un fichier

TABLE 5 – Taille maximale d'un fichier mesurée

Système	Taille max	Facteur limitant
VeraCrypt (2 Go)	1 859 Mo	Taille du volume
JFFS2 (256 Mo)	4 095 Mo	Limite théorique 32 bits (`dsize`)
UBIFS (256 Mo)	6 498 Mo	Volume UBI compressé (LZO)
F2FS (512 Mo)	395 Mo	Taille du volume

La valeur de 4 095 Mo pour JFFS2 confirme la limite théorique de 4 Go due au champ `dsize` encodé sur 32 bits, analogue à la limite FAT-32 évoquée dans le cours [1]. Pour UBIFS, 6 498 Mo dépasse la taille nominale du volume (256 Mo) car UBI utilise la compression LZO par défaut.

4.4 Overhead VeraCrypt selon la taille de cluster

TABLE 6 – Overhead VeraCrypt selon la taille de cluster interne (volume 512 Mo)

Cluster	Overhead	Espace disponible
1 024	≈0,00%	440 Mo
2 048	≈0,00%	440 Mo
4 096	≈0,00%	440 Mo

L'overhead est identique quelle que soit la taille de cluster car VeraCrypt remplit l'intégralité du volume avec des données aléatoires chiffrées dès la création. Ce comportement diffère radicalement de ce que décrit le cours [1] pour les systèmes classiques, où une taille de bloc plus grande entraîne davantage de gaspillage d'espace (fragmentation interne).

4.5 Performances séquentielles selon la taille de blocs

TABLE 7 – Performances en écriture séquentielle (MB/s)

Système	512B	4K	64K	1M
VeraCrypt	59,5	93,4	93,9	80,2
JFFS2	3,0	159	335	285
UBIFS	60,8	450	477	525
F2FS	52,1	78,5	67,8	69,5

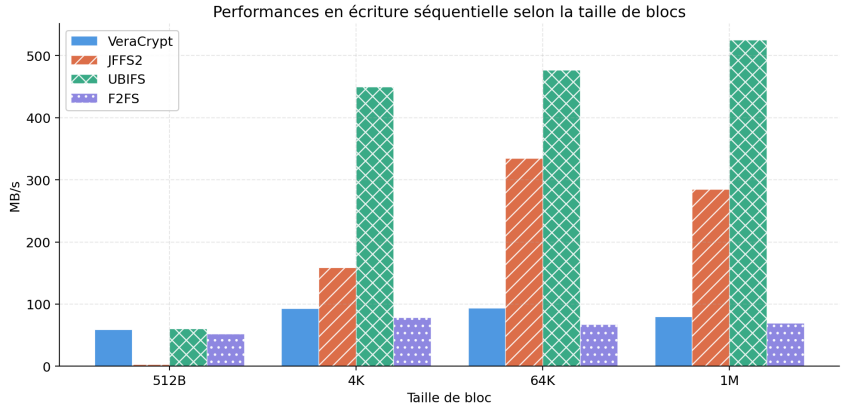


FIGURE 2 – Performances en écriture séquentielle selon la taille de blocs. L'écart JFFS2/UBIFS, particulièrement visible à partir de 4K, illustre l'impact du GC bloquant de JFFS2 vs le GC asynchrone d'UBIFS.

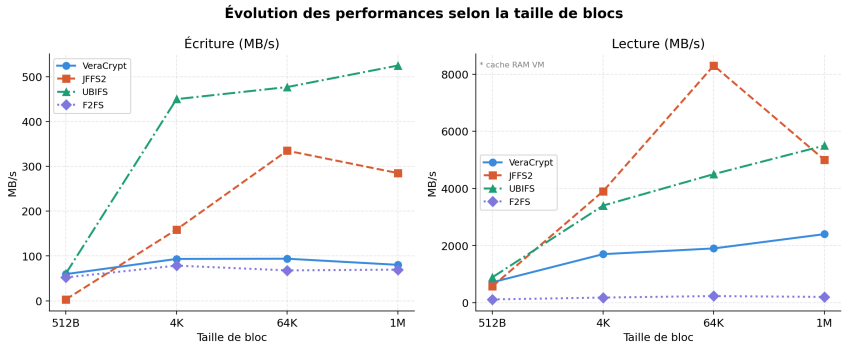


FIGURE 3 – Évolution des performances en écriture (gauche) et en lecture (droite) selon la taille de blocs. Les courbes mettent en évidence la progression quasi-linéaire d'UBIFS en écriture et le comportement atypique de JFFS2 en lecture (dû au cache RAM de la VM).

TABLE 8 – Performances en lecture séquentielle (MB/s)

Système	**512B**	**4K**	**64K**	**1M**
VeraCrypt	716	1 700	1 900	2 400
JFFS2	573	3 900	8 300	5 000
UBIFS	890	3 400	4 500	5 500
F2FS	115	181	233	203

TABLE 9 – Débit de création de fichiers (5 000 fichiers)

Système	**Débit (fichiers/s)**
VeraCrypt	296
JFFS2	446
UBIFS	466
F2FS	440

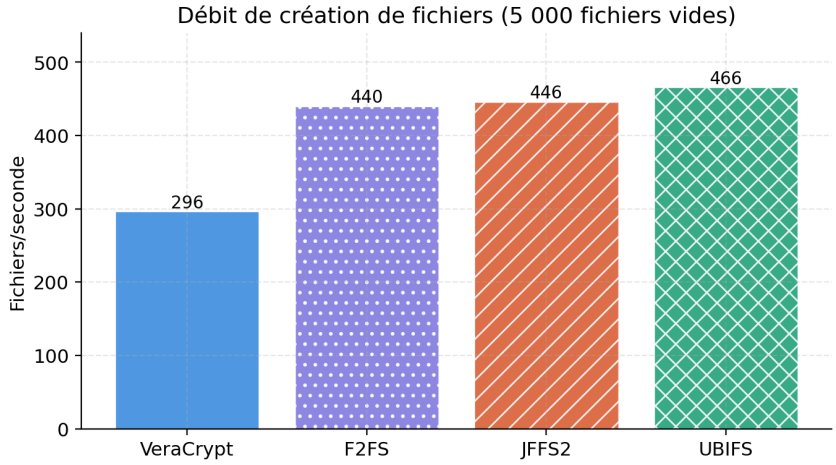


FIGURE 4 – Débit de création de 5 000 fichiers vides. VeraCrypt est le moins performant car chaque création de fichier implique l'allocation d'un i-node et une opération de chiffrement AES pour les métadonnées.

5 Discussion

5.1 Analyse des performances d'écriture

JFFS2 vs UBIFS. Le résultat le plus parlant est l'écart d'écriture entre JFFS2 (3,0 MB/s à 512B) et UBIFS (60,8 MB/s à 512B), soit un facteur de 20. Pour les grands blocs (1M), UBIFS atteint 525 MB/s contre 285 MB/s pour JFFS2. Cet écart s'explique par les mécanismes de journalisation présentés dans le cours [1] : JFFS2 effectue une journalisation complète (*full journaling*) bloquante après chaque écriture, tandis qu'UBIFS implémente un cache d'écriture en RAM (*write-back*) et un GC asynchrone en arrière-plan, approche analogue au cache d'écriture décrit dans le cours, qui améliore les performances au prix d'une fenêtre de vulnérabilité en cas de crash. Ces résultats sont cohérents avec ceux d'Achour [3] qui mesure un facteur 2,5× en écriture séquentielle sur Fedora natif.

VeraCrypt. Les performances d'écriture de VeraCrypt (59,5–93,9 MB/s) sont comparables à celles d'UBIFS en petits blocs. Le chiffrement AES-256 en mode XTS ajoute un overhead CPU d'environ 10–20% par rapport à un accès direct. Cet overhead est plus visible à 512B car le mode XTS opère secteur par secteur (taille minimale d'un bloc disque selon le cours [1]), déclenchant une opération de chiffrement indépendante par secteur.

F2FS. Les performances de F2FS (52–79 MB/s en écriture) sont inférieures aux attentes car F2FS est optimisé pour les SSD réels avec parallélisme de canaux ; son architecture multi-tête n'apporte pas de bénéfice sur un fichier image séquentiel en VM.

5.2 Analyse des performances de lecture

Les performances de lecture très élevées (JFFS2 : 8,3 GB/s à 64K ; UBIFS : 5,5 GB/s à 1M) dépassent largement les capacités physiques du disque. Cette anomalie s'explique par le cache disque détaillé dans le cours [1] : Linux utilise *toute la mémoire libre disponible* comme cache de lecture, et même après invalidation via `echo 3 > /proc/sys/vm/drop_caches`, VirtualBox maintient une partie des données en mémoire hôte. Nos mesures de lecture reflètent donc la vitesse de déchiffrement/décompression en RAM plutôt que la vitesse disque réelle. F2FS présente des lectures plus basses (115–233 MB/s) car son architecture implique davantage de déréférencement de pointeurs (tables NAT/SIT), réduisant l'efficacité du *read-ahead* décrit dans le cours [1].

5.3 Comparaison avec les résultats d'Achour

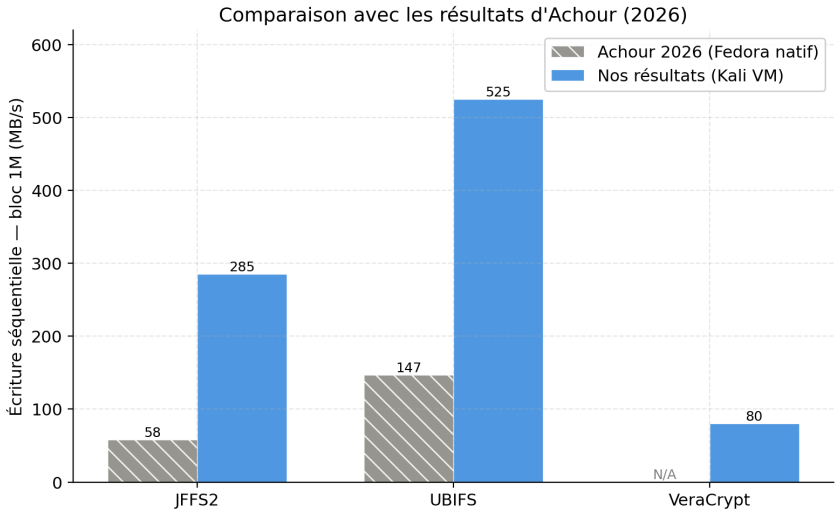


FIGURE 5 – Comparaison des débits d'écriture séquentielle (bloc 1M) entre les résultats d'Achour [3] sur Fedora natif et nos mesures en VM Kali.

Achour mesure sur Fedora natif un débit de création de 278 fichiers/s pour JFFS2 et 312 fichiers/s pour UBIFS. Nos résultats (446 f/s et 466 f/s) sont supérieurs car nos tests sont effectués sur un volume vide, alors que JFFS2 dégrade ses performances au fur et à mesure que le volume se remplit, comportement lié à l'augmentation du coût du *garbage collection* quand les blocs libres deviennent rares.

5.4 Vue d'ensemble comparative

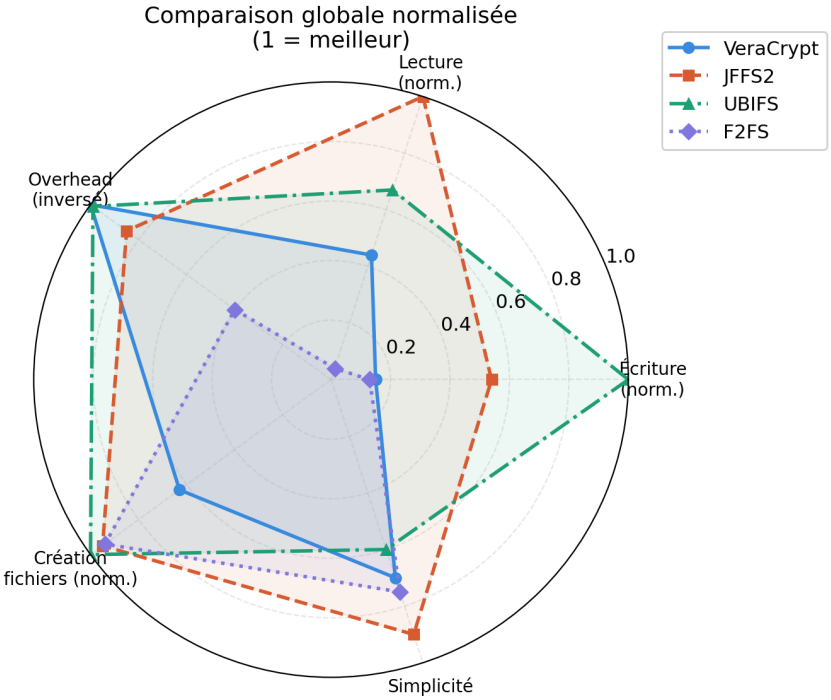


FIGURE 6 – Radar chart de comparaison globale normalisée (1 = meilleur dans chaque catégorie). UBIFS domine en écriture et création de fichiers. VeraCrypt offre l'overhead le plus faible mais pâtit du coût du chiffrement. F2FS présente le profil le plus équilibré.

5.5 Positionnement de VeraCrypt parmi les systèmes de fichiers

VeraCrypt ne relève pas de la définition d'un système de fichiers donnée dans le cours [1] : il n'organise pas les données sur le disque, ni ne gère les répertoires ou les permissions. C'est une couche orthogonale qui peut s'empiler sur n'importe quel FS. Son overhead de chiffrement est acceptable pour la sécurité qu'il apporte : un volume VeraCrypt est indéchiffrable sans la clé, même avec un accès physique au support. En revanche, son incompatibilité avec TRIM et la neutralisation des algorithmes de compression FTL en font un choix à peser soigneusement pour les SSD haute performance.

6 Conclusion

Cette étude confirme que le choix d'un système de fichiers pour support flash ne peut pas se réduire à une comparaison de débits bruts. Chaque système répond à un contexte précis : UBIFS pour les systèmes embarqués NAND exigeants, JFFS2 pour les petits volumes NOR, F2FS pour le mobile, et VeraCrypt comme couche de sécurité orthogonale applicable à n'importe lequel d'entre eux. Les systèmes obsolètes (JFFS, YAFFS2, LogFS) illustrent qu'une conception architecturale solide ne suffit pas sans maintenance active, leçon valable bien au-delà des systèmes de fichiers.

6.1 Synthèse des contributions

Ce travail apporte quatre contributions par rapport au rapport de référence d'Achour [3] :

1. **Analyse et benchmarks de VeraCrypt** : architecture interne (XTS, PBKDF2, header chiffré), impact sur les SSD (TRIM, FTL), et mesures de performances comparées aux systèmes MTD.

2. **Test de l'overhead selon la taille de cluster** : résultat inattendu, la taille de cluster n'affecte pas l'overhead visible, contrairement au comportement classique des FS décrits dans le cours [1].
3. **Mesure du nombre maximal de fichiers et répertoires** : révèle la limite d'inodes ext4 dans VeraCrypt (131 060), validant expérimentalement le mécanisme d'i-nodes du cours [1].
4. **Variation systématique de la taille de blocs** : confirme que le choix de la taille de bloc est déterminant, comme le souligne le cours [1], JFFS2 présente un facteur 100× entre 512B et 64K en écriture.

6.2 Recommandations

Table 10 – Recommandations par cas d'usage

Cas d'usage	Recommandation	Justification
Embarqué NAND > 64 Mo	UBIFS	Performance, faible RAM
Embarqué NOR < 64 Mo	JFFS2	Simplicité, pas d'UBI
Mobile / SSD flash	F2FS	Optimisé flash
Données sensibles	VeraCrypt + ext4	Chiffrement fort
Systèmes à éviter	JFFS, YAFFS2, LogFS	Obsolètes / non maintenus

6.3 Perspectives

Les travaux futurs pourraient inclure des tests sur matériel physique (SSD NVMe ou flash embarquée réelle) pour s'affranchir des biais de virtualisation, ainsi que l'évaluation de l'impact réel de VeraCrypt sur l'endurance d'un SSD via des mesures du nombre de cycles d'écriture.

Annexe A - Script de benchmark

La fonction principale illustrée ci-dessous correspond au test de performances séquentielles (Test 5), qui constitue le coeur du protocole expérimental.

```
#!/bin/bash
# Benchmark generique - Usage : sudo bash benchmark.sh /mnt/
    point_de_montage
MP="$1"
LOG=~/resultats/$(basename $MP).txt
echo "=== BENCHMARK $(date) ===" | tee $LOG

# Overhead disque a vide
df -B1 $MP | tail -1 | awk \
  '{printf "Overhead: %.2f%%\n", $3*100/$2}' | tee -a $LOG

# Performances selon taille de blocs (Test 5)
for BS in 512 4096 65536 1048576; do
  COUNT=$(( 32*1024*1024/BS ))
  W=$(dd if=/dev/zero of=$MP/perf bs=$BS count=$COUNT \
    conv=fdatasync 2>&1 | grep -oP '[\d,.]+ [MkG]B/s' | tail -1)
  sudo sh -c 'echo 3 > /proc/sys/vm/drop_caches'
  R=$(dd if=$MP/perf of=/dev/null bs=$BS 2>&1 | \
    grep -oP '[\d,.]+ [MkG]B/s' | tail -1)
  echo "bs=$BS Ecriture:$W Lecture:$R" | tee -a $LOG
  rm -f $MP/perf
done
```

Références

[1] Sid Touati, *C3-FileSystems - File systems, Advanced Operating Systems*, Université Côte d'Azur, Master 1 Informatique, 2025–2026.

[2] Sid Touati, *C2-MemoryManagement - Memory Management, Advanced Operating Systems*, Université Côte d'Azur, Master 1 Informatique, 2025–2026.

[3] Djerada Achour, *Étude Comparative des Systèmes de Fichiers pour SSD*, Université Côte d'Azur – INRIA, Janvier 2026.

[4] David Woodhouse, *JFFS2 : The Journalling Flash File System, version 2*, Ottawa Linux Symposium, 2001.

[5] Artem Hunter et al., *A Brief Introduction to the Design of UBIFS*, Nokia, 2008.

[6] Changman Lee et al., *F2FS : A New File System for Flash Storage*, USENIX FAST, 2015.

[7] Charles Manning, *How YAFFS Works*, `https://yaffs.net`, 2012.

[8] IDRIX, *VeraCrypt Documentation*, `https://veracrypt.io/en/Documentation.html`, 2024.

[9] Jörn Engel, *LogFS – finally a scalable flash filesystem*, Ottawa Linux Symposium, 2005.